Lecture 18: Transport Layer, UDP, and Reliable Data Transfer

EC 441 — Introduction to Computer Networking

Boston University, Spring 2026

Learning Objectives

By the end of this lecture, you should be able to:

1. Explain what the transport layer provides that the network layer does not, and describe how multiplexing and demultiplexing work using ports and sockets.
2. Describe the UDP header and explain the role of the UDP checksum.
3. Explain why UDP is the right choice for certain applications, and give concrete examples.
4. Explain why reliable data transfer over an unreliable channel requires more than just retransmission.
5. Analyze the link utilization of Stop-and-Wait and explain why pipelining is necessary on high-bandwidth-delay-product paths.
6. Describe how Go-Back-N and Selective Repeat work and trace their behavior through a packet loss event.
7. Explain the key design tradeoff between Go-Back-N and Selective Repeat, and identify which ideas TCP inherits from each.

1 The Transport Layer

Up through Lecture 17, we have been working at the *network layer*: IP moves datagrams from one host to another on a best-effort basis. The network layer knows only about hosts — it addresses packets to machines. But processes, not machines, send and receive data. A single host may simultaneously run a web browser, an SSH client, a video call, and a background software update. All four generate and receive packets. The **transport layer** closes this gap by providing *process-to-process* communication on top of the host-to-host service that IP provides.

In the **5-layer Internet model** (also called the TCP/IP model), the transport layer is layer 4, sitting between the network layer (layer 3, IP) and the application layer (layer 5). The **OSI model** subdivides the application layer into three separate layers — Session (5), Presentation (6), and Application (7) — giving seven layers total. In practice, these distinctions matter little for TCP/IP: TLS/SSL spans the session and presentation roles, and most application-layer protocols bundle everything together. The five-layer model is the standard framing used in this course.

Layer	Name (5-layer)	Protocol Data Unit
5	Application	Message / stream
4	**Transport**	**Segment (TCP) / Datagram (UDP)**
3	Network	Datagram (IP)
2	Link	Frame
1	Physical	Bits

The handoff between layers is explicit in the IPv4 datagram header: the one-byte **Protocol** field tells the receiving host which transport-layer protocol to deliver the payload to. TCP is Protocol 6; UDP is Protocol 17. (ICMP is Protocol 1, as seen in Lecture 17.) When a packet arrives, the IP layer strips the IP header and hands the payload to whichever transport protocol the Protocol field names.

1.1 Multiplexing and Demultiplexing

The transport layer at the sender must **multiplex**: it gathers data from many application sockets, adds transport-layer headers (including port numbers), and passes segments down to the network layer. At the receiver, the transport layer must **demultiplex**: it examines incoming segments, reads the destination port number, and routes each segment to the correct socket so that the right application process receives it.

Ports are 16-bit unsigned integers that identify a transport-layer endpoint on a host. They are divided into three ranges by IANA:

- **Well-known ports (0–1023)**: assigned by IANA to specific services. A process must have elevated privileges to bind a well-known port.
- **Registered ports (1024–49151)**: assigned by IANA on request; can often be used without elevated privileges.
- **Ephemeral ports (49152–65535)**: assigned dynamically by the OS for the client side of a connection.

Table 1 lists several well-known ports that appear frequently in this course.

Table 1: Selected well-known port numbers

Port	**Protocol**	**Service**
22	TCP	SSH (Secure Shell)
53	TCP/UDP	DNS (Domain Name System)
67	UDP	DHCP (server)
68	UDP	DHCP (client)
80	TCP	HTTP
123	UDP	NTP (Network Time Protocol)
443	TCP	HTTPS (HTTP over TLS)

1.2 Sockets and the 5-Tuple

The operating system represents a transport-layer endpoint as a **socket**: an abstraction that applications use to send and receive data. Every socket is identified by a **5-tuple**:

$$(\text{protocol},\ \text{local IP},\ \text{local port},\ \text{remote IP},\ \text{remote port})$$

This 5-tuple matters for *demultiplexing*. For UDP, the demultiplexing key is the pair (destination IP, destination port): two UDP datagrams arriving at the same destination port from different sources land in the same socket, because UDP has no concept of a connection. For TCP, the full 5-tuple is the demultiplexing key. This is what makes it possible for a web server to maintain thousands of simultaneous TCP connections all arriving at port 443: each connection has a different source IP or source port, so each maps to a distinct socket and hence a distinct file descriptor and application handler.[1,2,3]

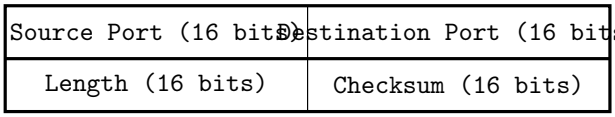
2 UDP

2.1 Design Philosophy

IP provides best-effort delivery at the network layer: it makes no promises about whether datagrams arrive, arrive in order, or arrive only once. **UDP (User Datagram Protocol)** extends that philosophy up one layer. Reliability, ordering, duplicate detection, and flow control are the *application's problem*, not the protocol's. This is a deliberate design choice, not an oversight. For applications that need tight latency control, that perform their own retransmission, or that use UDP as a building block for higher-level transports, the absence of protocol-level mechanisms is an advantage.

2.2 The UDP Header

The UDP header is 8 bytes, consisting of four 16-bit fields.



- **Source port**: identifies the sending socket; used by the receiver to reply.
- **Destination port**: the demultiplexing key at the receiver.
- **Length**: length in bytes of the UDP header plus data (minimum 8).
- **Checksum**: error detection over the segment; discussed below.

[1] Wikipedia contributors. "Transport layer." Wikipedia. https://en.wikipedia.org/wiki/Transport_layer

[2] Wikipedia contributors. "Port (computer networking)." Wikipedia. https://en.wikipedia.org/wiki/Port_(computer_networking)

[3] Wikipedia contributors. "Network socket." Wikipedia. https://en.wikipedia.org/wiki/Network_socket

Why does UDP have a Length field but TCP does not?

The IP header already records the total datagram length, so the UDP Length field is technically redundant — yet it is there by design, for two reasons.

1. Checksum self-containment. The UDP checksum is computed over a pseudo-header (source IP, destination IP, protocol, *UDP length*) concatenated with the UDP header and data. Having the length explicitly in the UDP header means the checksum can be computed and verified without consulting the IP layer. Removing it would tightly couple UDP's integrity check to IP.

2. Optional checksum. In IPv4, setting the checksum field to `0x0000` disables it entirely. When the checksum is omitted, the UDP Length field is the *only* in-protocol source of payload size that does not require reading the surrounding IP header.

TCP makes the opposite choice (RFC 793 §3.1): it derives its payload length directly from the IP layer — `data = IP.TotalLength − IP.HeaderLen − TCP.HeaderLen` — because TCP was always designed as an integral part of the TCP/IP protocol suite and never needs to stand alone. A useful way to remember the asymmetry: UDP carries its own framing so it can be checksum-verified independently of IP; TCP delegates framing to IP entirely.

What UDP does *not* provide: delivery guarantee, ordering, duplicate detection, congestion control, or connection setup. Its entire on-wire overhead is 8 bytes, compared to 20–60 bytes for a TCP header.[4]

2.3 The UDP Checksum

The UDP checksum is computed using ones'-complement addition over a **pseudo-header** — a synthetic structure that includes the source IP address, destination IP address, a zero byte, the protocol number (17), and the UDP length — followed by the UDP header and data padded to an even number of bytes. Including IP addresses in the checksum guards against *misdelivered datagrams*: a packet that arrives at the right port but was forwarded to the wrong host due to a corrupted IP header will fail the UDP checksum even if the UDP header itself is intact.

This is the same ones'-complement principle as the IPv4 header checksum: the minimum Hamming distance for the checksum code is $d_{\min} = 2$, meaning all single-bit errors are detected (connecting back to the error detection discussion in Lectures 5–6).

In IPv4, the checksum is *optional*: a value of `0x0000` indicates that no checksum was computed. In IPv6, the UDP checksum is *mandatory*, because the IPv6 header itself carries no checksum.

2.4 When UDP Is the Right Choice

Because UDP imposes almost no overhead, it is preferable to TCP in several important situations:

- **Low-latency interactive applications**: VoIP and real-time games need fresh data fast. A retransmitted voice packet that arrives 200 ms late is useless; it is better to discard it and play out silence.

[4] Wikipedia contributors. "User Datagram Protocol." Wikipedia. `https://en.wikipedia.org/wiki/User_Datagram_Protocol`

- **Query-response protocols**: DNS, NTP, and DHCP send a single request and expect a single reply. The overhead of a TCP handshake (at least one full RTT before any data) would double the latency of a name lookup.
- **Multicast and broadcast**: TCP is a point-to-point protocol; it cannot multicast. Streaming protocols that send to a group must use UDP.
- **Application-controlled reliability**: QUIC (the transport underlying HTTP/3) and WebRTC build their own reliability and congestion control on top of UDP, giving them fine-grained control that TCP's kernel implementation cannot provide. WireGuard tunnels traffic over UDP to avoid the *TCP-over-TCP pathology* — if a TCP session runs inside a TCP tunnel, the outer TCP's retransmissions interact badly with the inner TCP's, collapsing throughput. UDP tunnels avoid this entirely.

Demo

The following commands illustrate UDP in the wild.

```
# Show open UDP sockets and the processes that own them
ss -u -p

# DNS lookup using UDP (default)
dig +short bu.edu

# Force DNS over TCP (used for large responses or zone transfers)
dig +tcp bu.edu

# Query Google's TXT records (may exceed 512 bytes -- triggers TCP fallback)
dig TXT google.com
```

Why DNS sometimes uses TCP: The original DNS specification limited UDP responses to 512 bytes. Responses that exceed this limit set the *truncation* (TC) bit, signaling the resolver to retry the query over TCP. Modern resolvers also advertise a larger UDP payload size using the EDNS0 extension (typically 4096 or more bytes), which defers the TCP fallback for most responses. DNSSEC responses, however, frequently exceed even the EDNS0 advertised size and reliably trigger TCP. Zone transfers (`AXFR`) always use TCP.

3 Reliable Data Transfer

The rest of this lecture examines how to build reliable data transfer on top of an unreliable channel — the fundamental problem that TCP must solve, and whose solution we study through a sequence of progressively more capable protocols.

3.1 The Problem

An unreliable channel can do two bad things to packets: it can **corrupt bits** (which a checksum can detect) and it can **lose packets entirely** (which a checksum cannot help with, because there is nothing to check). A reliable data transfer (RDT) protocol must handle both failure modes.

The building blocks available to us are:

- **Checksums**: detect bit errors in received packets.

- **Acknowledgments (ACKs)**: positive feedback from receiver to sender, confirming receipt.
- **Sequence numbers**: integers attached to packets so the receiver can detect duplicates and reorder out-of-order deliveries.
- **Timers**: the sender starts a timer when it transmits; if no ACK arrives before the timer expires, the sender retransmits.

The challenge is combining these correctly *and* efficiently. Getting correctness alone is harder than it looks.

4 Stop-and-Wait

The simplest ARQ (Automatic Repeat reQuest) protocol is **Stop-and-Wait**, also called the Alternating Bit Protocol. The rule is elementary: the sender transmits one packet and then waits for an ACK before transmitting the next. The receiver sends an ACK for every correctly received packet.

4.1 Sender and Receiver State Machines

The sender alternates between two states:

1. **Wait for call from above (seq 0)**: accept data from the application, send packet with sequence number 0, start timer, move to "wait for ACK 0".
2. **Wait for ACK 0**: if ACK 0 arrives (uncorrupted), stop timer, move to "wait for call from above (seq 1)". If timer fires or a corrupted/wrong ACK arrives, retransmit, restart timer.

The same pair of states exists for sequence number 1. Only a 1-bit sequence number is needed because the protocol keeps at most one packet in flight at a time: the receiver only ever needs to distinguish "is this the packet I am expecting next" from "is this a retransmit of the packet I already delivered."

4.2 Failure Scenarios and Why Sequence Numbers Are Necessary

Three things can go wrong, and each must be handled correctly.

Case 1 — Data packet lost. The timer fires, the sender retransmits, and the receiver eventually delivers the packet and sends an ACK. This is clean: no ambiguity arises because the receiver never saw the first attempt.

Case 2 — ACK lost. The data packet is delivered correctly, the receiver sends an ACK, but the ACK is lost in the network. The sender's timer fires and it retransmits the same packet. Now consider what happens at the receiver: it has already delivered this data to the application. Should it deliver it again?

Without sequence numbers, the receiver has no way to answer this question. The arriving packet looks identical to the original. If the receiver blindly delivers it, the application receives the data twice. This is a *correctness failure*.

With a 1-bit sequence number, the receiver can reason as follows: "I just delivered a packet with sequence number 0 and sent ACK 0. I am now waiting for a packet with sequence number 1.

This arriving packet has sequence number 0. Therefore it is a retransmit of something I already delivered. I will discard the data, but I will re-send ACK 0 so the sender knows to move on." The application receives the data exactly once.

Case 3 — Corrupted packet. The checksum detects the corruption. The receiver sends a NAK (or, equivalently, re-sends the ACK for the last correctly received packet), and the sender retransmits.

Key Result

The core correctness requirement for any ARQ protocol: the receiver must be able to distinguish new data from a retransmit. This is why sequence numbers are necessary even when a single-bit label is sufficient.

4.3 Link Utilization Under Stop-and-Wait

Stop-and-Wait is correct, but it can be extraordinarily inefficient. Consider the following example.

Stop-and-Wait Utilization on a Satellite Link

Suppose a link has bandwidth $B = 1$ Gb/s and round-trip time RTT $= 600$ ms (a realistic figure for a geostationary satellite link). Packet size is 1500 bytes $= 12{,}000$ bits.

The transmission time for one packet is:

$$t_{\text{tx}} = \frac{12{,}000 \text{ bits}}{10^9 \text{ bits/s}} = 12 \ \mu\text{s}$$

Under Stop-and-Wait, the sender transmits for 12 μs and then idles for the remaining $\approx$ 600 ms waiting for the ACK. Link utilization is:

$$U = \frac{t_{\text{tx}}}{\text{RTT} + t_{\text{tx}}} = \frac{12 \times 10^{-6}}{600 \times 10^{-3} + 12 \times 10^{-6}} \approx 0.002\%$$

A 1 Gb/s link delivers approximately **20 kb/s** of useful throughput under Stop-and-Wait. The link is idle more than 99.998% of the time.

Key Result

Stop-and-Wait link utilization:

$$U = \frac{t_{\text{tx}}}{\text{RTT} + t_{\text{tx}}}$$

When RTT $\gg t_{\text{tx}}$ — which is common on high-bandwidth or long-distance links — utilization collapses to near zero.

5 Bandwidth-Delay Product and Pipelining

The quantity BDP $= B \times \text{RTT}$ is the **bandwidth-delay product**: the number of bits that can be "in flight" on the link at once.[5] For our satellite example:

$$\text{BDP} = 1 \text{ Gb/s} \times 600 \text{ ms} = 600 \text{ Mb}$$

Stop-and-Wait keeps exactly *one packet* in flight, which is 12,000 bits out of 600×10^6 bits of pipe capacity. To fully utilize the link, the sender would need to keep roughly:

$$W = \frac{\text{BDP}}{\text{packet size}} = \frac{600 \times 10^6 \text{ bits}}{12{,}000 \text{ bits/packet}} = 50{,}000 \text{ packets}$$

simultaneously in flight.

Pipelining is the solution: the sender maintains a **window** of up to N outstanding (sent but not yet acknowledged) packets. As long as the window is not full, the sender can continue transmitting without waiting for ACKs. This keeps the pipe filled.

Two important window-based protocols handle the case when a packet in the pipeline is lost. They make different tradeoffs between simplicity and efficiency.

6 Go-Back-N

In **Go-Back-N (GBN)**, the sender maintains a window of up to N unacknowledged packets.[6,7] The key design choices are:

- **Cumulative ACKs**: ACK_n means "I have received all packets through sequence number n correctly." The sender advances the window base to $n + 1$.
- **Single timer**: one timer covers the oldest unacknowledged packet. If it fires, the sender retransmits *all* packets currently in the window.
- **Receiver discard**: the receiver accepts packets only in order. If packet n arrives out of order, the receiver discards it and re-sends an ACK for the last in-order packet it received. No receiver buffering is required.

[5] Wikipedia contributors. "Bandwidth-delay product." Wikipedia. `https://en.wikipedia.org/wiki/Bandwidth-delay_product`

[6] Wikipedia contributors. "Go-Back-N ARQ." Wikipedia. `https://en.wikipedia.org/wiki/Go-Back-N_ARQ`

[7] Wikipedia contributors. "Sliding window protocol." Wikipedia. `https://en.wikipedia.org/wiki/Sliding_window_protocol`

Go-Back-N Trace with Packet Loss

Suppose window size $N = 4$ and sequence numbers are large enough to avoid ambiguity.

1. Sender transmits packets 0, 1, 2, 3 (filling the window).
2. Receiver gets packet 0, sends ACK 0; gets packet 1, sends ACK 1.
3. Packet 2 is *lost* in the network.
4. Receiver gets packet 3, but since it is out of order (expected 2), it **discards** packet 3 and re-sends ACK 1.
5. Sender receives duplicate ACK 1s. The timer for packet 2 eventually expires.
6. Sender **retransmits packets 2, 3, 4, 5** — all packets in the current window, not just the lost one.

One lost packet causes the sender to retransmit the entire outstanding window.

Sequence number constraint. With k-bit sequence numbers (giving 2^k distinct values), the GBN window size must satisfy:

$$N \leq 2^k - 1$$

One sequence number must be reserved so the receiver can unambiguously identify the boundary between the window and the "already acknowledged" region.

7 Selective Repeat

Go-Back-N's retransmit-the-whole-window policy can waste bandwidth when losses are rare but the window is large. **Selective Repeat (SR)** fixes this by having the receiver *buffer* out-of-order packets and the sender retransmit *only* the specific packet that was lost.[8]

The key design choices are:

- **Individual ACKs**: the receiver sends ACK_n for each correctly received packet n, whether or not it is in order.
- **Individual timers**: the sender maintains a separate timer for each outstanding packet.
- **Receiver buffer**: packets within the receiver's window are buffered until any gap below them is filled. When the in-order frontier advances, buffered packets are delivered to the application.
- **Duplicate handling**: if the receiver gets a packet whose sequence number falls below its current window (i.e., already delivered), it re-sends an ACK for it so the sender's retransmit timer can be cleared.

[8] Wikipedia contributors. "Selective Repeat ARQ." Wikipedia. `https://en.wikipedia.org/wiki/Selective_Repeat_ARQ`

Selective Repeat Trace with Packet Loss

Same scenario: window $N = 4$, packet 2 lost.

1. Sender transmits packets 0, 1, 2, 3.
2. Receiver gets packet 0, sends ACK 0; gets packet 1, sends ACK 1.
3. Packet 2 is *lost*.
4. Receiver gets packet 3, which is within the receive window. It **buffers** packet 3 and sends ACK 3.
5. Sender's timer for packet 2 expires. Sender retransmits **only packet 2**.
6. Receiver gets packet 2: it fills the gap, delivers packet 2 and the buffered packet 3 to the application in order, and slides its window forward.

One lost packet causes exactly one retransmission.

Sequence number constraint. For SR with k-bit sequence numbers:

$$N \leq 2^{k-1}$$

This constraint is stricter than GBN's. The window must be at most *half* the sequence number space. The reason: if the window were larger, the receiver could not distinguish between a new packet (from the next window) and a retransmit of a packet it has already delivered. With $N = 2^{k-1}$, new-window sequence numbers always differ by at least N from already-delivered ones, making the two ranges disjoint.

8 Go-Back-N vs. Selective Repeat, and the Road to TCP

8.1 Comparison

Table 2 summarizes the tradeoffs between the two protocols.

Table 2: Go-Back-N vs. Selective Repeat

Property	**Go-Back-N**	**Selective Repeat**
Receiver buffer	None (discard OOO)	Required (buffer OOO pkts)
Retransmit on loss	Entire window	Lost packet only
ACK type	Cumulative	Individual
Timers at sender	One (oldest unACKed)	One per outstanding packet
Seq. num. constraint	$N \leq 2^k - 1$	$N \leq 2^{k-1}$
Best when	Low loss, simple receiver	Higher loss, can buffer

8.2 What TCP Inherits

TCP is not a pure implementation of either protocol; it takes the best ideas from both. By default, TCP uses **cumulative ACKs** in the style of GBN: an ACK for byte offset n confirms receipt of all bytes up through n. This simplicity has always been part of TCP's base specification.

However, TCP requires a **receiver buffer** (as in SR): arriving segments are held until gaps below them are filled before the data is handed to the application. This is essential for performance: without buffering, every gap would force a full retransmit-the-window like GBN.

When the **Selective Acknowledgment (SACK)** TCP option is negotiated, the receiver can explicitly report which byte ranges it has already received. The sender uses this information to retransmit only the specific missing segments, exactly as in SR. SACK is supported by essentially all modern operating systems and is enabled by default in Linux, macOS, and Windows.

In summary: TCP's default behavior closely resembles GBN (cumulative ACKs, single retransmit timer), but with a receiver buffer and the SACK option it behaves more like SR. This hybrid design is one reason TCP performs well across such a wide range of network conditions.

8.3 What Comes Next

Stop-and-Wait, GBN, and SR answer the question: *how do we deliver data reliably?* But a complete transport protocol must also answer several more:

- **Flow control**: what if the receiver's buffer is full? The sender must slow down.
- **Congestion control**: what if the network itself is overloaded? The sender must slow down for a different reason.
- **Connection management**: how do two processes agree to start communicating, and how do they cleanly terminate?
- **Byte-stream semantics**: TCP is a stream protocol, not a message protocol. How does segment boundaries interact with application writes?

Lecture 19 addresses all of these and completes the TCP picture.